

DBI-Projektdokumentation: Finnldy

Projekt: Finnldy

Team: Muhammet Altuntas, Jakob Seeberger

Schuljahr: 2025/26

1. Projektbeschreibung

Finnldy ist eine datenbankgestützte REST-API für eine Gruppen-Filmempfehlungs-App. Die Grundidee ist, dass mehrere Benutzer Filme bewerten können, indem sie unterschiedliche Swipe-Arten vergeben. Dadurch kann später berechnet werden, welche Filme innerhalb einer Gruppe am besten passen.

Das Backend verwaltet Benutzer, Filme und Swipes. Zusätzlich gibt es Tabellen für Genres, die Zuordnung zwischen Filmen und Genres sowie weitere Filmdetails. Die API wurde mit FastAPI umgesetzt und verwendet SQLite als relationale Datenbank. Die Datenbankzugriffe werden mit SQLAlchemy durchgeführt.

Ziel des Projekts ist es, ein vollständiges Backend mit Datenbankmodell, CRUD-Endpunkten, Rollenrechten, Validierung, Logging sowie JOIN- und Aggregationsabfragen umzusetzen.

2. Ziel des Systems

Das System soll folgende Aufgaben erfüllen:

- Benutzer verwalten
 - Filme in der Datenbank speichern
 - Swipes von Benutzern zu Filmen speichern
 - Ergebnisse aus mehreren Swipes berechnen
 - Daten über eine REST-API als JSON bereitstellen
 - Schreibende Operationen nur für Admins erlauben
 - Lesende Operationen auch für normale User erlauben
-

3. Must-have und Nice-to-have Features

Must-have

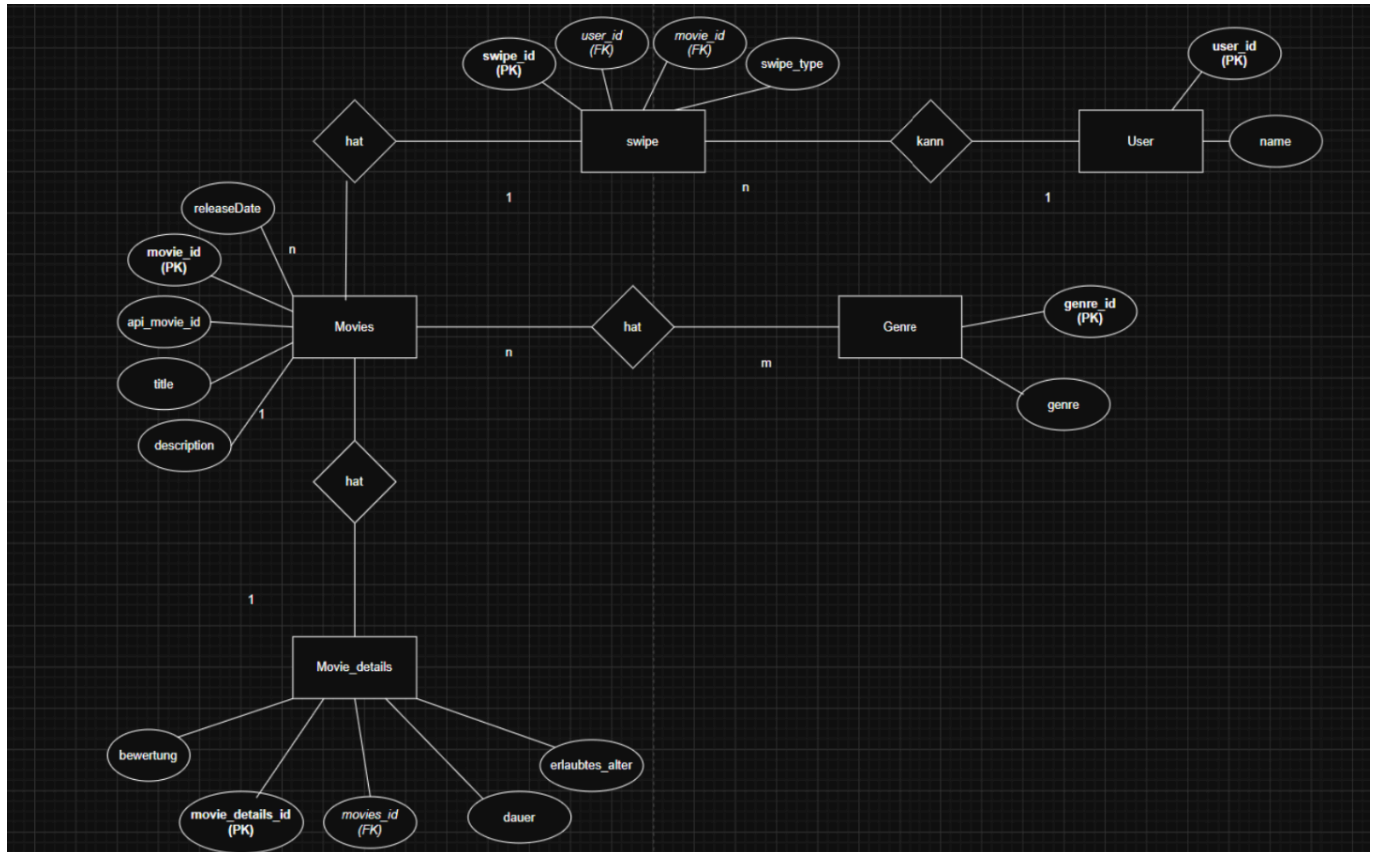
Feature	Status
Benutzer können erstellt, angezeigt, bearbeitet und gelöscht werden.	Erledigt
Filme und Swipes werden in der Datenbank gespeichert.	Erledigt
Die API gibt das Ergebnis der am besten bewerteten Filme aus.	Erledigt

Nice-to-have

Feature	Status
Filme können nach Genre, Erscheinungsjahr oder Bewertung gefiltert werden.	Erledigt
Eine Watchlist- oder Lobby-Funktion kann ergänzt werden.	Erledigt

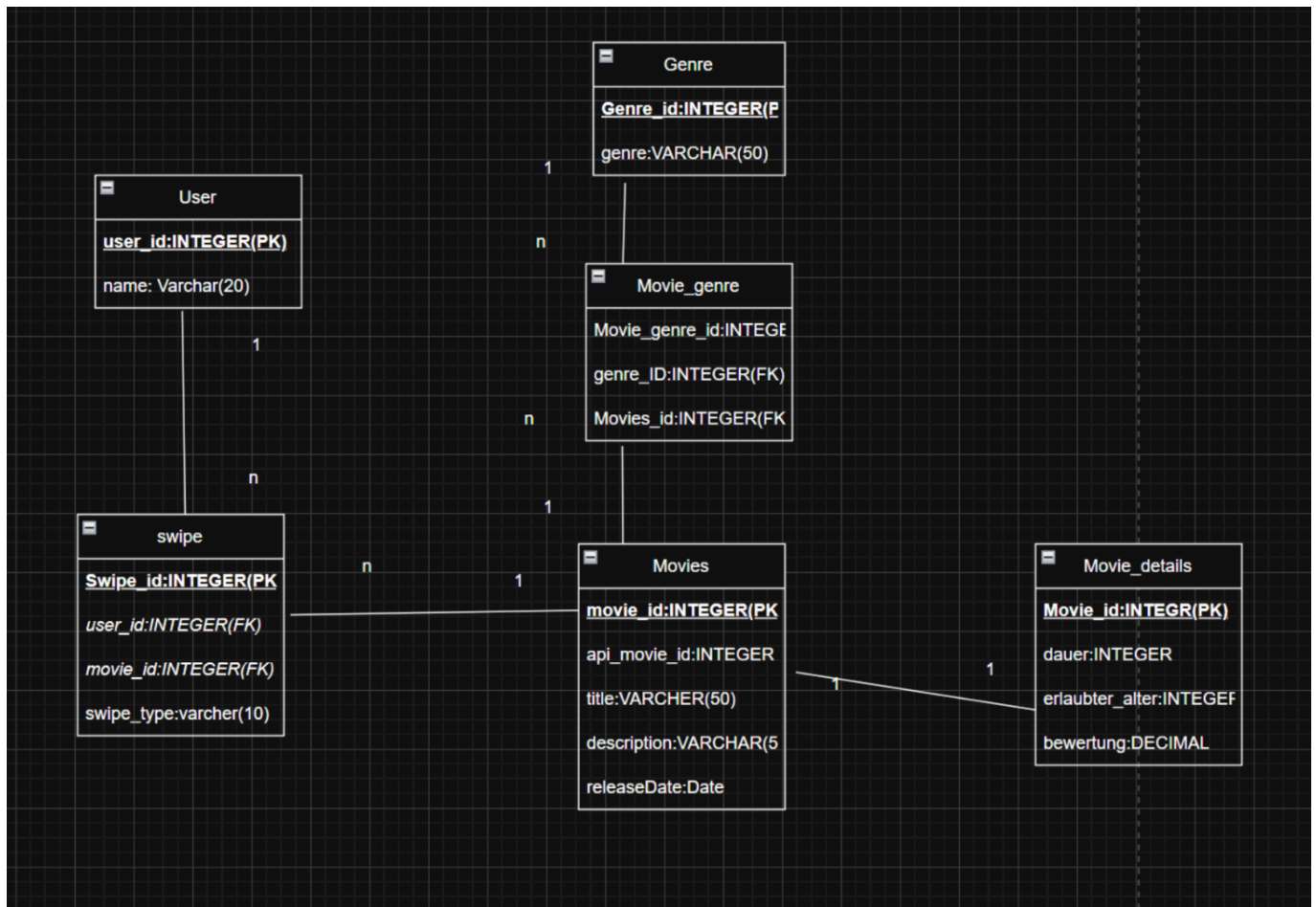
Die wurden im C# projekt gemacht

4. ERM



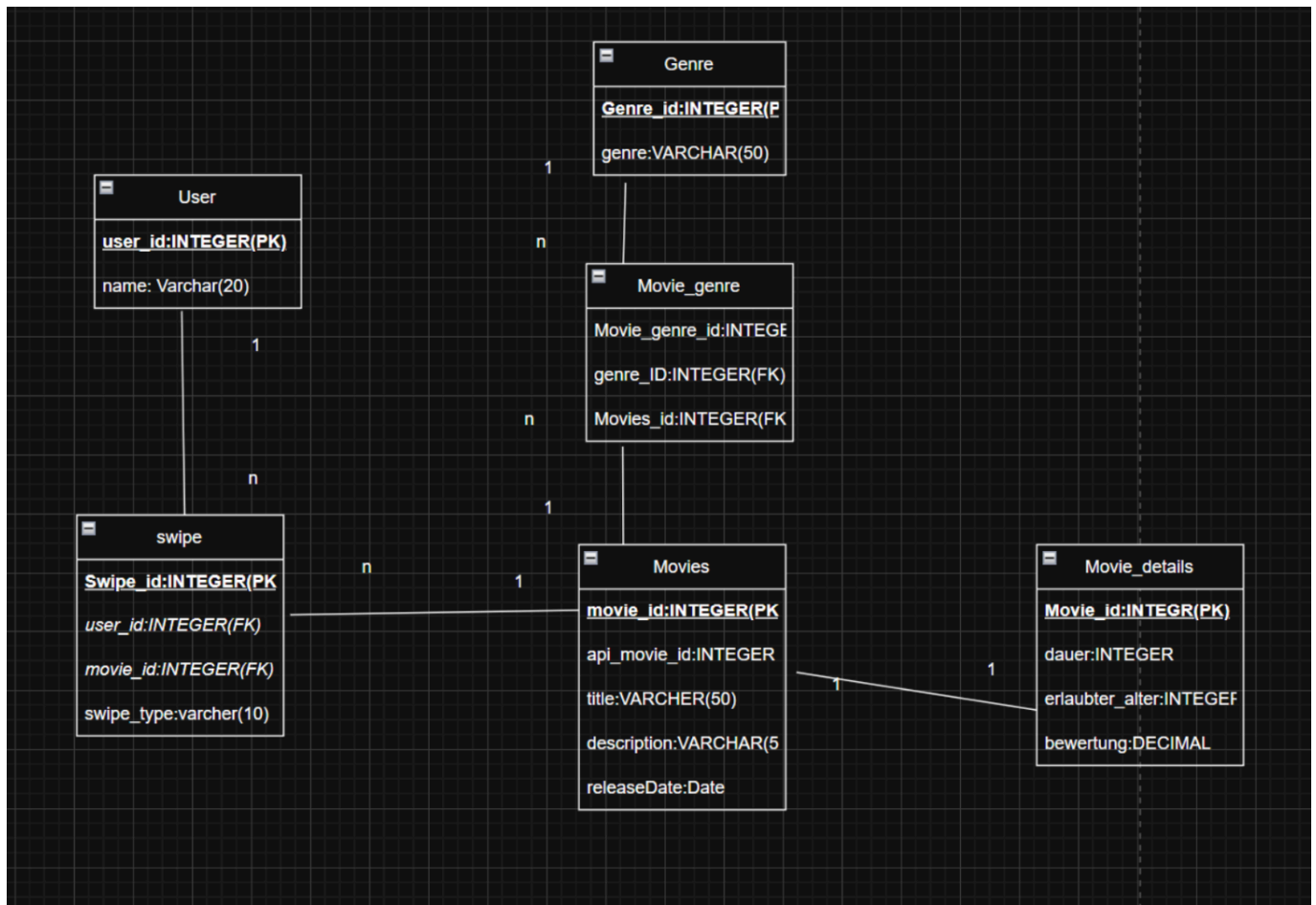
5. Normalisierung

1.NF



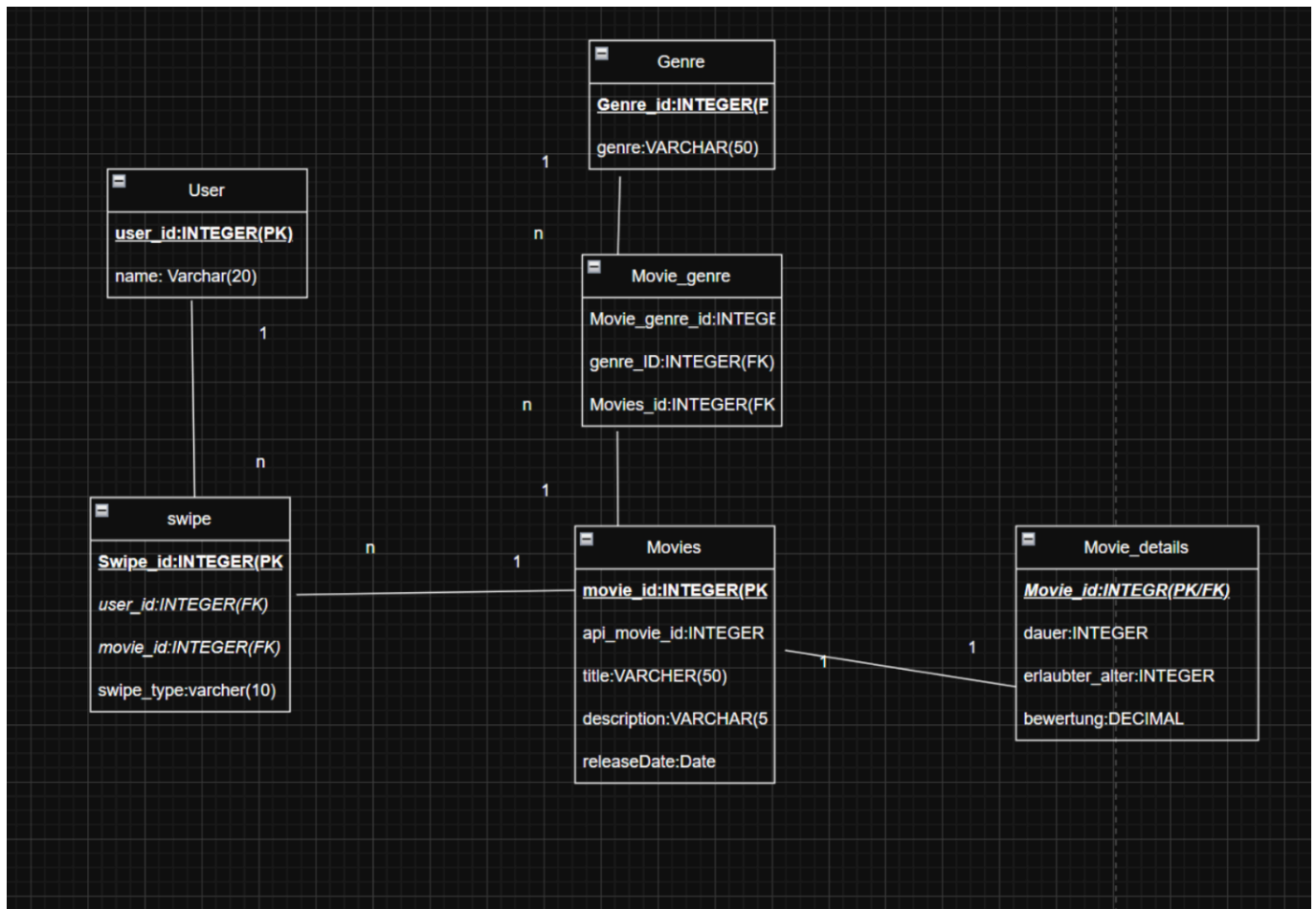
Passt musste man nichts ändern.

2.NF



Hat wieder gepasst.

3.NF



Movie_details (PK) wurde zu movie_id(FK).

6. API-Endpunkte

Methode	Pfad	Beschreibung	Rolle
GET	/	Test-Endpoint der API	keine / allgemein
GET	/users/	Gibt alle Benutzer zurück. Unterstützt limit und offset .	admin, user
GET	/users/{user_id}	Gibt einen einzelnen Benutzer anhand der ID zurück.	admin, user
POST	/users/	Erstellt einen neuen Benutzer.	nur admin
PUT	/users/{user_id}	Aktualisiert einen Benutzer vollständig.	nur admin
DELETE	/users/{user_id}	Löscht einen Benutzer.	nur admin
GET	/users/{user_id}/swipes	Gibt alle Swipes eines Benutzers zurück.	admin, user
POST	/swipes/	Erstellt einen neuen Swipe. Falls Benutzer oder Film noch nicht existieren, werden diese angelegt.	nur admin
GET	/swipes/results	Gibt eine berechnete Film-Rangliste zurück.	admin, user

7. CRUD-Nachweis

CRUD-Operation	Methode	Pfad	Beschreibung
Create	POST	/users/	Neuer Benutzer wird erstellt.
Read all	GET	/users/	Alle Benutzer werden aufgelistet.
Read one	GET	/users/{user_id}	Ein einzelner Benutzer wird angezeigt.
Update	PUT	/users/{user_id}	Ein Benutzer wird vollständig aktualisiert.
Delete	DELETE	/users/{user_id}	Ein Benutzer wird gelöscht.

8. JOIN und Aggregation

8.1 JOIN-Endpunkt

```
SELECT movies.id, movies.api_movie_id, movies.title
FROM movies
JOIN swipes ON swipes.movie_id = movies.id
```

8.2 Aggregationsendpunkt

```
SELECT movies.id, movies.title, COUNT(swipes.id)
FROM movies
JOIN swipes ON swipes.movie_id = movies.id
GROUP BY movies.id
```

9. Rollen und Rechte

Die API verwendet API-Keys im Header **X-API-Key**. Es gibt zwei Rollen:

API-Key	Rolle	Rechte
admin	Admin	Darf GET, POST, PUT und DELETE verwenden.
user	User	Darf nur lesende GET-Endpunkte verwenden.

10. Validierung und Fehlerbehandlung

10.1 Beispiele für Validierung

Feld	Regel
name	Muss 2 bis 20 Zeichen lang sein.

Feld	Regel
<code>name</code>	Darf nur Buchstaben und Leerzeichen enthalten.
<code>username</code>	Muss 2 bis 20 Zeichen lang sein.
<code>api_movie_id</code>	Muss größer als 0 sein.
<code>movie_title</code>	Muss 1 bis 50 Zeichen lang sein.
<code>swipe_type</code>	Darf nur <code>Like</code> , <code>Dislike</code> , <code>Watched</code> oder <code>WatchLater</code> sein.

10.2 Wichtige Statuscodes

Situation	Statuscode	Beispiel
Erfolgreiches GET	200	Benutzer wird gefunden.
Erfolgreiches POST	201	Benutzer oder Swipe wird erstellt.
Erfolgreiches PUT	200	Benutzer wird aktualisiert.
Erfolgreiches DELETE	200	Benutzer wird gelöscht.
Ressource nicht gefunden	404	Benutzer-ID existiert nicht.
Fehlender oder falscher API-Key	401	Kein gültiger <code>X-API-Key</code> .
Keine Berechtigung	403	User versucht POST/PUT/DELETE.
Ungültige Eingabe	422	Name zu kurz oder falscher Swipe-Typ.
Konflikt	409	Benutzername existiert bereits.

11. Bedienungsanleitung

11.1 Voraussetzungen

Für das Projekt wird Python benötigt. Danach müssen die Abhängigkeiten installiert werden.

11.2 Projekt installieren

Im Projektordner ausführen:

```
pip install -r requirements.txt
```

11.3 Datenbank initialisieren

```
python init_db.py
```

Dadurch werden die Tabellen aus den SQLAlchemy-Modellen erstellt.

11.4 API starten

```
uvicorn src.main:app --reload
```

Danach ist die API lokal erreichbar unter:

```
http://127.0.0.1:8000
```

Die Swagger-Dokumentation kann geöffnet werden unter:

```
http://127.0.0.1:8000/docs
```

11.5 API-Key verwenden

Für geschützte Endpunkte muss im Header ein API-Key mitgeschickt werden:

```
X-API-Key: admin
```

oder

```
X-API-Key: user
```

12. Aufgabenverteilung

mero

- users
- base
- swipes
- main
- doku
- init_db

Jakob

- logging
- auth
- rollenverteilung
- database
- models

13. Projekttagbuch

Datum	Person	Tätigkeit	Probleme / Lösungen
20.05.2026	Jakob	Erste Dateien und erste Dokumentationsentwürfe wurden ins Repository hochgeladen.	Erste Versionen der Dokumentation wurden später wieder entfernt und überarbeitet, da die Struktur noch nicht final war.
29.05.2026	Jakob	Git wurde getestet und mehrere Test-Commits wurden erstellt.	Ziel war es, zu prüfen, ob das gemeinsame Arbeiten mit Git funktioniert.
31.05.2026	mero	Erste <code>database.py</code> wurde erstellt, um die Datenbankbindung und Git-Funktionalität zu testen.	Es wurde geprüft, ob Dateien korrekt ins Repository übernommen werden.
08.06.2026	mero	Die erste <code>main.py</code> wurde erstellt und die FastAPI-Grundstruktur aufgebaut. Erste API-Punkte wurden vorbereitet.	Die API-Struktur war zu diesem Zeitpunkt noch nicht vollständig und musste später erweitert werden.
10.06.2026	Jakob	Die Datenbankverbindung mit SQLAlchemy wurde erstellt. Außerdem wurden die SQLAlchemy-Modelle für die Tabellen umgesetzt.	Die Tabellenstruktur wurde technisch in Python-Modelle übertragen.
11.06.2026	Jakob	Logging sowie die Rechte für die Rollen <code>admin</code> und <code>user</code> wurden begonnen.	Es wurde festgelegt, dass <code>admin</code> Schreib- und Löschrechte besitzt und <code>user</code> nur lesend zugreifen darf.
16.06.2026	Jakob	Logging und Rollenrechte wurden fertiggestellt. Zusätzlich wurde der Code auf SQL-Injection-Sicherheit überprüft.	Datenbankzugriffe wurden über SQLAlchemy umgesetzt, damit keine unsicheren SQL-Strings verwendet werden.
16.06.2026	mero	Router-Endpunkte wurden erstellt und <code>main.py</code> wurde angepasst.	Die API wurde in einzelne Router aufgeteilt, damit die Struktur übersichtlicher ist.
16.06.2026	Jakob	Ein Merge-Konflikt wurde behoben, nachdem das Programm durch den Konflikt nicht mehr korrekt funktioniert hatte.	Die beschädigte Struktur wurde wieder korrigiert und lauffähig gemacht.
17.06.2026	Team	Die Projektstruktur wurde für die Abgabe angepasst. Dateien wurden in einen <code>src</code> -Ordner verschoben und <code>init_db.py</code> wurde ergänzt.	Die API soll laut Vorgabe mit <code>uvicorn src.main:app</code> startbar sein. Dafür mussten Imports und Struktur angepasst werden.
17.06.2026	Team	Die API wurde über Swagger UI getestet. Dabei wurden unter anderem	Ein 500-Fehler trat auf, weil unterschiedliche SQLite-Dateien

Datum	Person	Tätigkeit	Probleme / Lösungen
		/users/, Rollenrechte und Datenbankzugriffe überprüft.	verwendet wurden. Die Lösung war, den Datenbankpfad eindeutig festzulegen und die Datenbank neu zu initialisieren.
17.06.2026	Team	Die finale Abgabe wurde vorbereitet: ZIP-Struktur, requirements.txt, Dokumentation und Startdateien wurden kontrolliert.	Unnötige Dateien wie __pycache__ und IDE-Dateien sollten vor der finalen Abgabe entfernt werden.
21.06.2026	mero	paar bugfixes behoben, init_db gemacht, doku fertig gestellt, paar sachen KI gekennzeichnet, und geschaut dass git in richtiger ordnung ist.	Kleine Fehler gewesen war nicht so schwer sie zu finden.

14. Reflexion

Im Projekt war positiv, dass die Grundstruktur der REST-API umgesetzt werden konnte und die Datenbank über SQLAlchemy angebunden wurde. Die Aufteilung in Router macht den Code übersichtlicher. Außerdem wurden Rollenrechte und Logging ergänzt, wodurch die API näher an einer realistischen Backend-Struktur ist.

Schwieriger war die Abstimmung zwischen Datenbank, Projektstruktur und Imports. Besonders beim Umstellen auf die src-Struktur mussten mehrere Imports angepasst werden. Außerdem musste darauf geachtet werden, dass die API die richtige SQLite-Datenbank verwendet.

Wenn das Projekt weiterentwickelt wird, könnte man eine echte externe Movie-API anbinden, mehr Filterfunktionen einbauen und die Tabellen genres, movie_genres und movie_details stärker in die API integrieren.